# Computing Fundamentals EC-110

Lecture # 05

# OBJECTIVE

- Understand the relationship between Boolean logic and digital computer circuits.
- Learn how to design simple logic circuits.
- Understand how digital circuits work together to form complex computer systems.

# BOOLEAN ALGEBRA

- *Boolean algebra is a mathematical system* for the manipulation of variables that can have one of two values.
  - *In formal logic*, these values are "true" and "false."
  - *In digital systems*, these values are "on" and "off," 1 and 0, or "high" and "low."
- Boolean expressions are created by performing operations on Boolean variables.
  - *Common Boolean operators* include *AND, OR,* and *NOT*.

# BOOLEAN ALGEBRA

- A Boolean operator can be completely described using a truth table.
- The *truth table for the Boolean operators AND and OR are shown at the right*.
- *The AND operator is also known as a Boolean product. The OR operator is the Boolean sum*.

X AND Y

| X | Y | XY |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

X OR Y

| X | Y | X+Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

# BOOLEAN ALGEBRA

- The truth table for the Boolean *NOT operator is shown at the right*.
- The *NOT operation is most often designated by an overbar*. It is sometimes indicated by a prime mark ( ‘ ) or an “elbow” (¬).

NOT x

| x | $\overline{x}$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

# BOOLEAN ALGEBRA

- A *Boolean function* has:
  - At *least one Boolean variable*,
  - At *least one Boolean operator*, and
  - At *least one input from the set {0,1}*.
- It *produces an output that is also a member of the set {0,1}*.

**Now you know why the binary numbering system is so handy in digital systems.**

# BOOLEAN ALGEBRA

- The *truth table for the Boolean function:*

$$F(x,y,z) = x\bar{z}+y$$

is shown at the right.

- To make evaluation of the Boolean function easier, the truth table contains extra (shaded) columns to hold evaluations of subparts of the function.

**$F(x,y,z) = x\bar{z}+y$**

| x | y | z | $\bar{z}$ | $x\bar{z}$ | $x\bar{z}+y$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 1 |

# BOOLEAN ALGEBRA

- *Most Boolean identities have an AND (product) form as well as an OR (sum) form*. We give our identities using both forms. Our first group is rather intuitive:

| Identity Name | AND Form | OR Form |
|---|---|---|
| Identity Law | $1x = x$ | $0 + x = x$ |
| Null Law | $0x = 0$ | $1 + x = 1$ |
| Idempotent Law | $xx = x$ | $x + x = x$ |
| Inverse Law | $x\bar{x} = 0$ | $x + \bar{x} = 1$ |

# BOOLEAN ALGEBRA

- Our *second group of Boolean identities* should be familiar to you from your study of algebra:

| Identity Name | AND Form | OR Form |
|---|---|---|
| Commutative Law | $xy = yx$ | $x+y = y+x$ |
| Associative Law | $(xy)z = x(yz)$ | $(x+y)+z = x+(y+z)$ |
| Distributive Law | $x+yz = (x+y)(x+z)$ | $x(y+z) = xy+xz$ |

# BOOLEAN ALGEBRA

- Our last group *(second group)* of Boolean identities are *perhaps the most useful*.
- If you have studied set theory or formal logic, these laws are also familiar to you.

| Identity Name | AND Form | OR Form |
|---|---|---|
| Absorption Law | $x(x+y) = x$ | $x + xy = x$ |
| DeMorgan's Law | $\overline{(xy)} = \bar{x} + \bar{y}$ | $\overline{(x+y)} = \bar{x}\bar{y}$ |
| Double Complement Law | $\overline{(\bar{x})} = x$ | |

# BOOLEAN ALGEBRA

- Sometimes it is more economical to build a circuit using the complement of a function (and complementing its result) than it is to implement the function directly.
- *DeMorgan's law provides an easy way of finding the complement of a Boolean function*.
- Recall DeMorgan's law states:

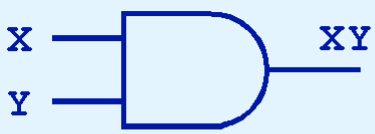
$$\overline{(xy)} = \bar{x} + \bar{y} \quad \text{and} \quad \overline{(x+y)} = \bar{x}\bar{y}$$

# LOGIC GATES

- We have looked at Boolean functions in abstract terms.
- *In this section, we see that Boolean functions are implemented in digital computer circuits called gates.*
- *A gate is an electronic device that produces a result based on two or more input values.*
  - In reality, gates consist of one to six transistors, but digital designers think of them as a single unit.
  - Integrated circuits contain collections of gates suited to a particular purpose.

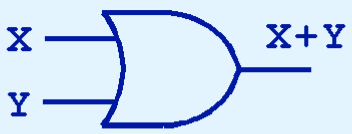
# LOGIC GATES

- The three simplest gates are the AND, OR, and NOT gates.



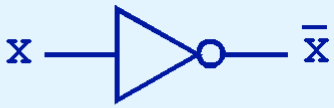

X AND Y

| X | Y | XY |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |




X OR Y

| X | Y | X+Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

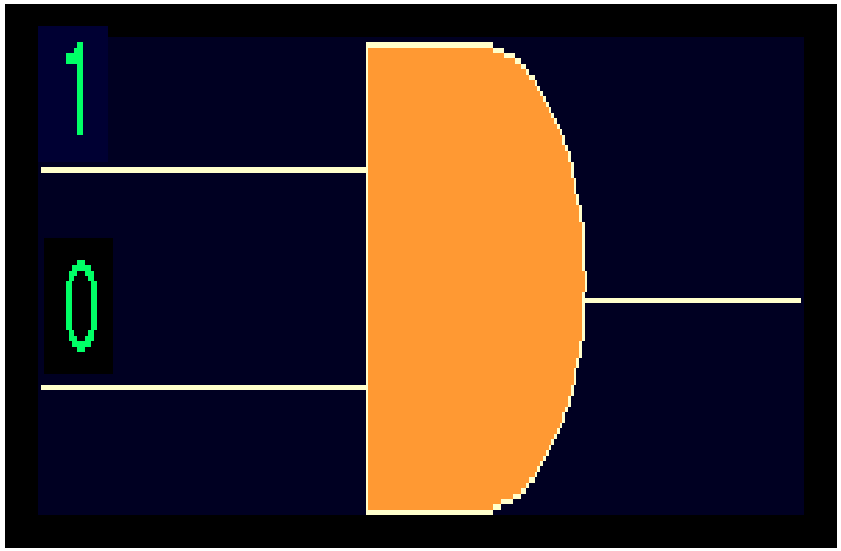



NOT X

| X | $\overline{X}$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

- They correspond directly to their respective Boolean operations, as you can see by their truth tables.

# LOGIC GATES





AND GATE

1 AND 0 = 0

OR GATE

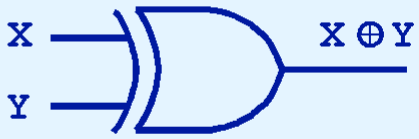
1 OR 0 = 1

# LOGIC GATES

- *Another very useful gate is the exclusive OR (XOR) gate*.
- The output of the *XOR operation is true only when the values of the inputs differ*.

X XOR Y

| X | Y | X⊕Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

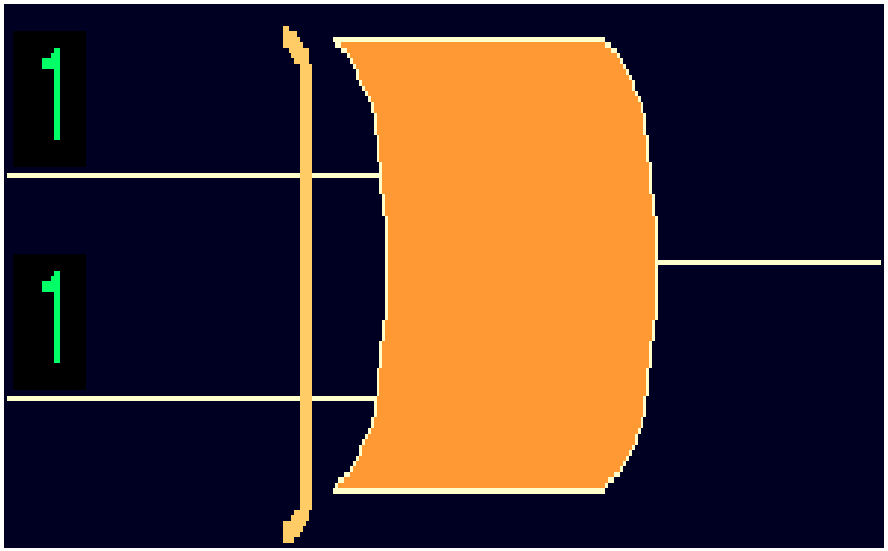



**Note the special symbol ⊕ for the XOR operation.**

# LOGIC GATES


XOR GATE

1 XOR 0 = 1




XOR GATE

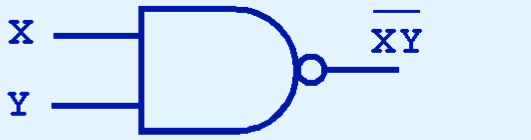
1 XOR 1 = 0

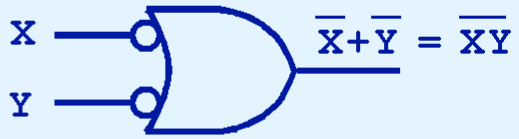
# LOGIC GATES

- *NAND and NOR are two very important gates*.
- *They are also called as Universal gates because we can implement any digital circuit using only these gates*.
- Their symbols and truth tables are shown at the right.

**X NAND Y**

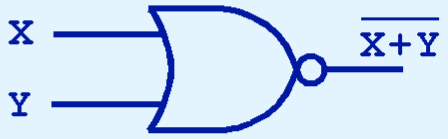
| X | Y | X NAND Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

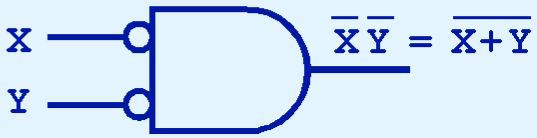


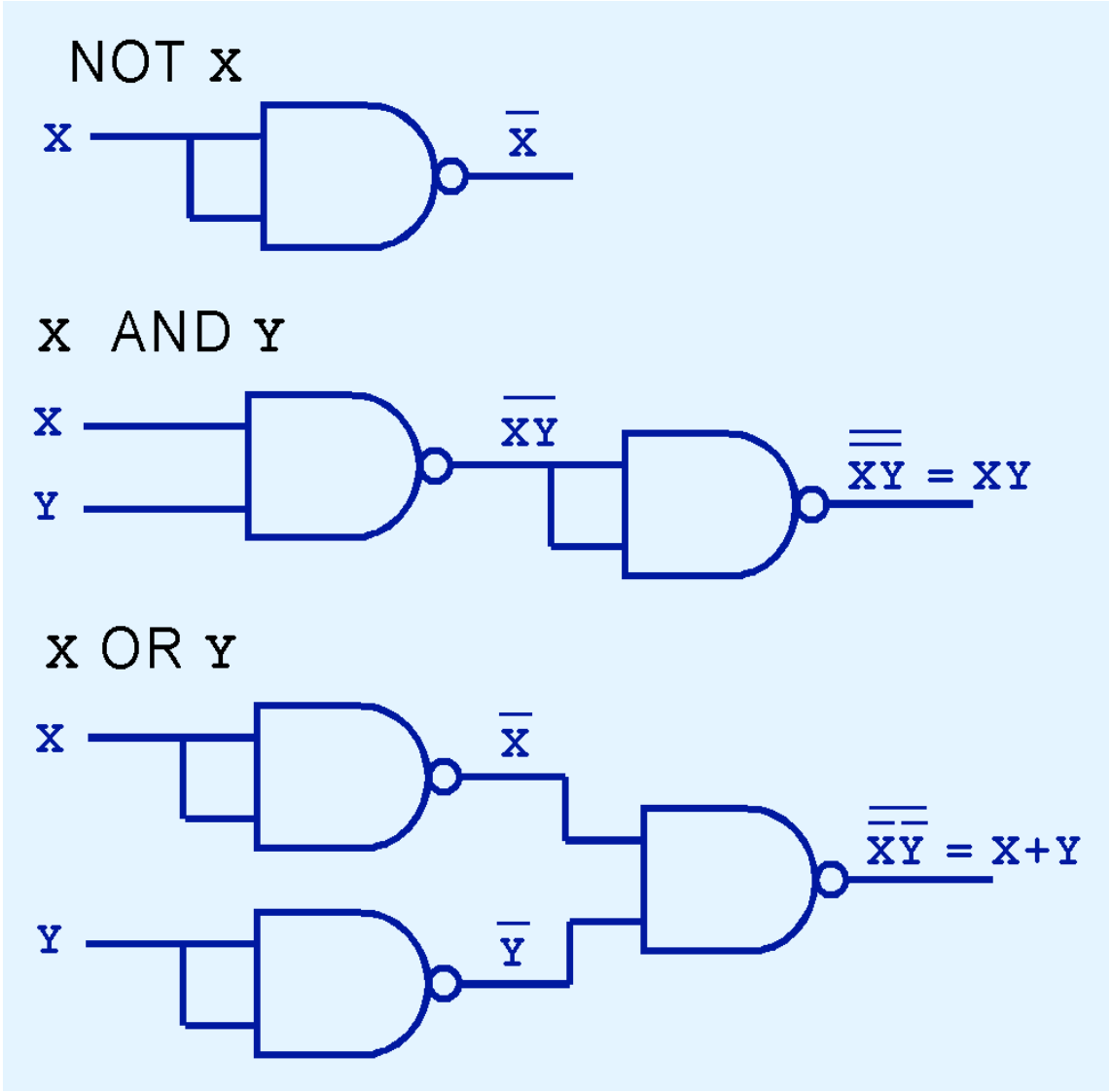

**X NOR Y**

| X | Y | X NOR Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |



# LOGIC GATES

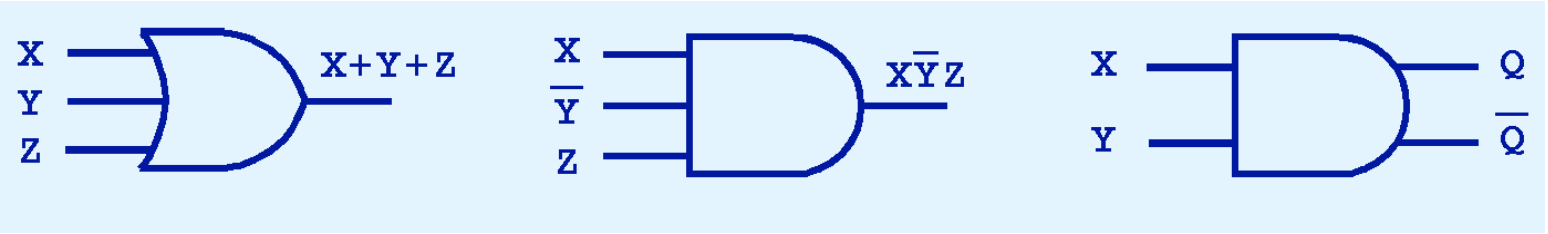



***NAND gate as an AND gate, OR gate and a NOT gate.***

***How about implementing AND gate, Or gate and a NOT gate using NOR gate?***

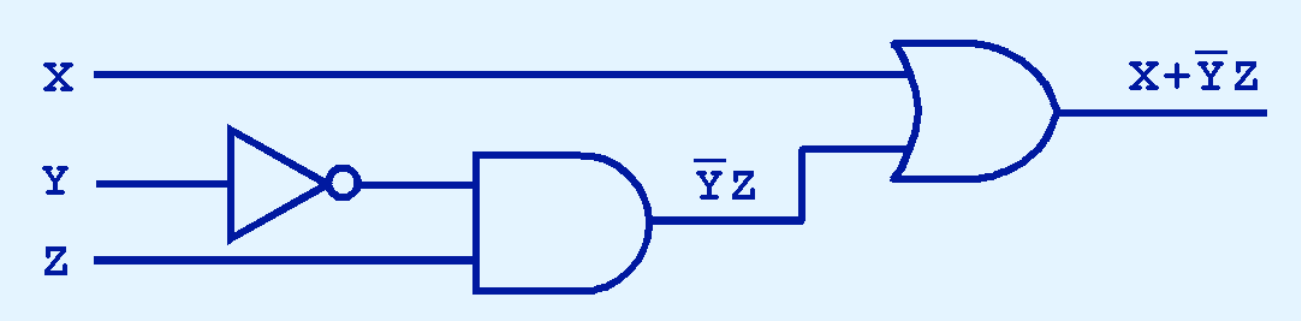
# LOGIC GATES

- *Gates can have multiple inputs and more than one output*.
  - A *second output can be provided for the complement of the operation*.
  - We'll see more of this later.



# DIGITAL COMPONENTS

- The *main thing to remember is that combinations of gates implement Boolean functions*.
- The circuit below implements the Boolean function: $F(X, Y, Z) = X + \overline{Y}Z$



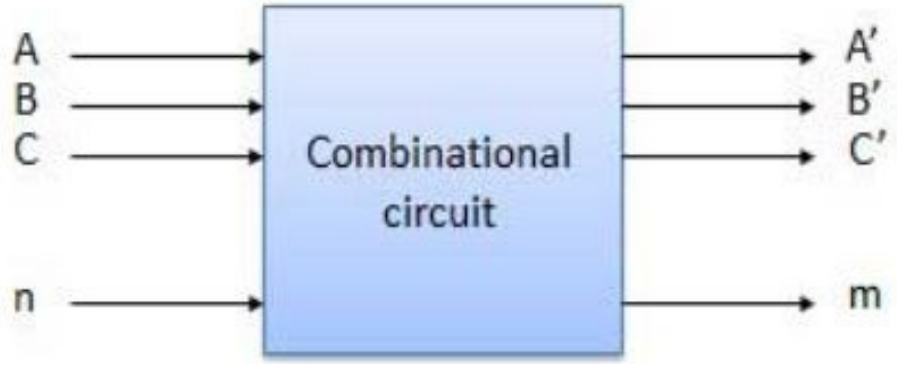

**We simplify our Boolean expressions so that we can create simpler circuits.**

# COMBINATIONAL CIRCUITS

- We have designed a circuit that implements the Boolean function:

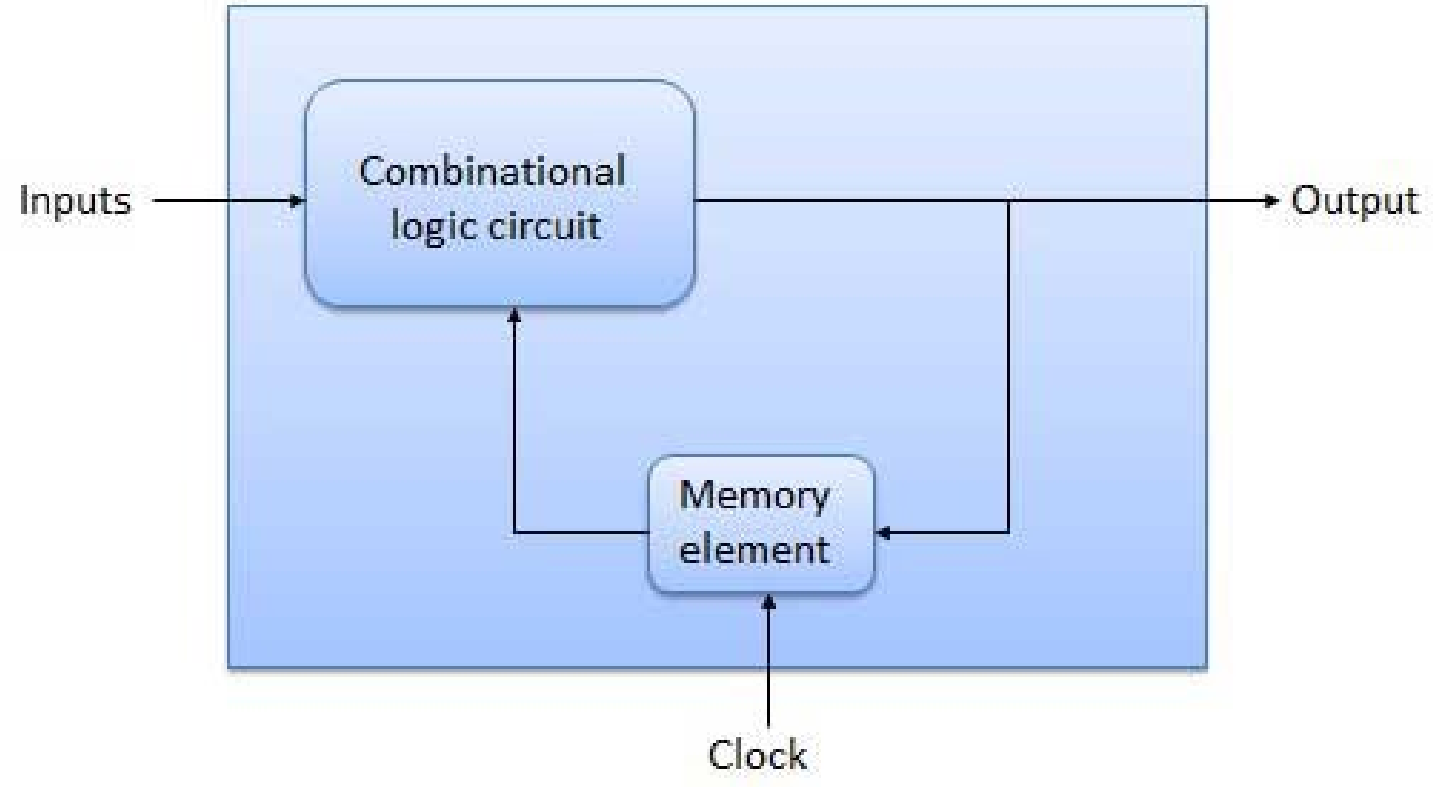
$$F(X,Y,Z) = X + \overline{Y}Z$$

- This circuit is an example of a *combinational logic* circuit.
- *Combinational logic circuit contains logic gates where its output is determined by the combination of the current input, regardless of the output or the prior combination of inputs*.



# SEQUENTIAL CIRCUITS

- *The combinational circuit does not use any memory. Hence the previous state of input does not have any effect on the present state of the circuit. But sequential circuit has memory so output can vary based on input. This type of circuits uses previous input, output, clock and a memory element.*



# CONCLUSION

- Computers are implementations of Boolean logic.
- Boolean functions are completely described by truth tables.
- Logic gates are small circuits that implement Boolean operators.
- The basic gates are AND, OR, and NOT.
  - The XOR gate is very useful in parity checkers and adders.
- The “universal gates” are NOR, and NAND.

# CONCLUSION

- Computer circuits consist of combinational logic circuits and sequential logic circuits.
- Combinational circuits produce outputs (almost) immediately when their inputs change.
- Sequential circuits require clocks to control their changes of state.